Prosaic v2: standby agents, adversarial review, and a faster loop

Proposal for discussion. Branch `dev`, checkout `~/code/prosaic_dev`. Nothing here is merged.

September 6, 2026

1. What thirty days of transcripts say

I parsed every Claude Code transcript on this machine from August 6 to September 6 (898 MB, 308 sessions, 82 subagent runs, 45 active days) and read the sync logs, launchd plists, settings and knowledge files of the live deployment.

Measure	Value
Interactive turns / active agent time	1,411 turns / 100 hours
Median turn, p90 turn (active seconds)	96 s / 633 s
Turns over 5 minutes	325 turns carrying 73 of the 100 hours
Tokens: cache read / cache write / output / thinking	6.4 B / 125 M / 16.4 M / 3.8 M
List-price equivalent	about $6,200 per month; 64 % of it is cache reads
Context size per API call (median, p90)	380 k / 800 k tokens; 47 % of calls above 400 k
Sessions spanning more than 36 hours	20 of 52, carrying 79 % of all calls
Effort level	`high` on 31,512 calls, `medium` on 101
Turns that delegated to a subagent	47 of 1,411 (3 %), mostly to Fable or Opus
Routine prompts (build, open, commit, push)	53 turns, median 39 s, p75 146 s
Scheduled triage runs	34, median 3.9 min, about 20 calls each, on the 1M Fable model with no model, turn or budget cap

The slowness is context, not model. Every call re-reads a 400k to 800k token prefix. Prefill on that much cache is seconds per call before any thinking starts, and a routine three-call build turn pays it three times. The long-lived sessions are the cause: twenty sessions that ran for days carry four fifths of all traffic. Switching Fable for Sonnet would not fix this. Starting sessions fresh, keeping instructions small, and pushing bulk reading into subagents would.

Routine commands are cheap when they stay routine. A bare “rebuild” takes 39 seconds at the median. The long tail is the model quietly fixing a source file when the build warns, which is real work wearing a routine prompt. The fix is a command path that runs the CLI and stops, so fix work becomes a separate, visible decision.

Effort is never tuned. Nearly every call ran at `high`. Thinking was 23 % of all output. There is no per-task effort or model routing anywhere in prosaic or in the harness settings.

The automation that exists is half-blind. Twice-daily sync and triage run and work, but connector failures are logged as `ok` because a `sed` in the pipeline masks the exit status. One connector has been broken for three weeks without a signal. Triage runs the most expensive model with permissions bypassed and no cap.

Knowledge has outgrown its rule. The largest matter’s KNOWLEDGE.md is 3,556 lines, about 82k tokens, with fourteen date-titled sections appended as a diary, two “as of August 13” snapshots superseded by later entries, zero cross-links and no tooling to notice any of it. TODO.md is 1,555 lines for 41 items.

2. Audit verdict on prosaic

The core bet holds: model drafts Markdown, deterministic code renders it, one seam (`cli/agent-run`) reaches whatever headless agent is installed, flows (`flows/run.py`) chain agent, command, judge and gate steps with a human approval file. That is the right skeleton for everything below. The gaps:

- **No model or role selection.** `agent-run` picks a CLI, never a model. Flows have no per-step provider. The judge, triage, and drafting all run on the one default.
- **The adversarial pass exists as a seed and is opt-in.** `flows/draft-review.yaml` already does opposing-counsel review, revise, judge, gate. Nothing runs it before `--final` or `sc sign`, there is no cite check, no clerk or bench persona, and no data source for how opposing counsel actually behaves.
- **No hooks, no slash commands, no shipped user skills.** The only agent-facing surface is prose SKILL.md files, two of which fail the repo's own 120-line test, so the suite is red.
- **Deployment is ahead of upstream.** The private deployment carries ADR-0038, `sc build-doc`, the build manifest, read-coverage triage and revised skills that prosaic lacks. The prosaic-first rule is currently inverted.
- **Push is blocked.** The pre-push leak guard rejects three files and one commit message on `main` for a form-number pattern, so eighteen local commits and the new `dev` branch cannot reach GitHub until scrubbed.
- **Dead and stale.** `prosaic/` holds only `.pyc` leftovers; `sc clean` judges by config, not age, so 70 files older than 30 days in `out/` are invisible to it.

3. Proposal, ranked by payoff per week of work

W0. Unblock and realign (small, first). Scrub the form-number fingerprints, push `main` and `dev`. Port the deployment-only changes up into prosaic so the deployment is a pure downstream again. Fix the two over-long skills, delete the dead package directory. Nothing else should land until `dev` pushes clean.

W1. Fast path for routine work (small, largest felt-speed win). Ship `.claude/commands/` from the matter template: `/build`, `/rebuild`, `/open`, `/commit`, `/clean`, `/status`. Each runs the CLI, prints stderr warnings verbatim, and stops; a warning that needs a source edit is reported, not fixed. Route them to a cheap model. Add a `SessionStart` hook that injects a short matter brief (parties, posture, next dates, top TODO items) instead of relying on the model to read the whole knowledge file. Adopt one session per task. Expected: cache-read spend and per-call latency roughly halved, and "rebuild" becomes a ten-second command.

W2. Standby loops (medium). A `prosaic-supervisor` per matter, installed by `sc schedule`, replacing the two 12-hourly plists:

- *Inbox watcher.* launchd `WatchPaths` on `inbox/` and each connector's staging directory; triage starts within a minute of a file landing, with a model, turn cap and budget set per role.
- *Honest sync.* Fix the exit-status masking, write a run summary to `.state/`, surface failures in the daily brief.
- *Nightly knowledge refinement.* A flow that reads KNOWLEDGE, INDEX, MANIFEST and the day's commits, proposes integrated edits and a list of open questions, and stops at a gate. It never writes KNOWLEDGE unattended.
- *Retention.* `sc clean --older-than` for `out/` renders and `.flow/` run directories, still report-first, never touching `assets/`, `pleadings/` or `processed_files/`.
- *Daily standup.* A `/standup` command that opens a 30-minute interactive session over the refinement's questions plus QUESTIONS.md, writes answers into the right knowledge files, and commits as `record`. The nightly flow queues, you answer once a day.

W3. Pre-signature adversarial gate (medium). A `pre-signature.yaml` flow that `--final` builds and `sc sign` require, keyed to the source hash:

1. *Cite check* (deterministic first): extract every statute, rule and case citation; verify form and existence against a local authority cache, flag anything unverified for a human.
2. *Clerk persona:* filing compliance, form boxes, service, page limits, caption and exhibit rules.
3. *Skeptical bench persona:* what the judge will not believe, what is unsupported, what is asked for without authority.

4. *Opposing counsel persona*, briefed from an `oppo_profile.md` the refinement loop maintains from observed filings and correspondence: how they have attacked before, what they will move to strike, what they will say you omitted.
5. Human gate with the four reports side by side.

Each step declares its own provider and effort so personas can run on different models. Until W5 exists, "different model" means a different Claude model or a local model, not another vendor (see below).

W4. Knowledge as a directory (medium). An ADR replacing the single file with `knowledge/` topic files, each with front matter (`related:`, `updated:`, `sources:`), and KNOWLEDGE.md reduced to an index with one line per topic. A linter, `sc knowledge check`, enforces absolute dates, no date-titled sections, every file indexed, every link resolving, and flags topics untouched since a later docket event. Migration of the live matters is a gated agent task you approve file by file. The nightly refinement loop and the session brief both become cheap once knowledge is topical.

W5. Backend choice and the privilege boundary (large, staged).

- *Now:* an `agent:` block in deployment config and `matter.yaml` naming provider, model and endpoint per role (triage, judge, clerk, bench, oppo, drafting). `agent-run --role` reads it. Document the self-hosted path: Claude Code or Codex CLI pointed at an OpenAI-compatible endpoint serving Qwen, GLM, Kimi or gpt-oss. Every role defaults to local or first-party; nothing goes to a second vendor by default.
- *Later:* a privilege column in INDEX.md set at intake (none, attorney-client, work product, psychotherapist-patient, medical, other); an entity map built from `matter.yaml` and INDEX; a `sanitize` flow step that produces a pseudonymized working copy in the run directory; a hard runner rule that a step whose provider is marked `external` receives only sanitized inputs; and an append-only log of what left the machine and how it was transformed.

A caution on W5. Pseudonymization reduces exposure; it does not by itself preserve privilege, and whether a given disclosure waives anything is a legal judgment, not a tooling property. The system's job is to make the boundary explicit, default to local, and leave an audit trail. I would not send drafts or privileged records to a second vendor, even for the opposing-counsel pass, until the sanitizer exists and you have decided the legal question.

3a. What it should save: an estimate

These are list-price equivalents computed from the same month of transcripts, holding the work constant and changing only how it is run. They are estimates; the mechanisms are measured, the mix is assumed.

Lever	Assumption	Saving per month
Fresh sessions, session brief, subagents for bulk reading (W1, W2)	Context per call capped near 150k instead of a 380k median	about $2,500 (41 %)
Reviewer-directed small changes on Opus, Fable for drafts (decision 2)	Half of current Fable calls move to Opus	about $900 (15 %)
Cheap models and capped effort for triage and routine commands (W1, W2)	Sonnet or Haiku with turn and budget caps	about $150 (2 %)
Combined	Levers overlap, so less than the sum	about $3,000 to $3,300 (50 %), from $6,150 to roughly $3,000

Thinking tokens are not a cost lever: all 3.8 M of them cost about $125. Effort tuning is worth doing for latency, not for money.

Speedup. Measured on this month's calls, median API latency rises with context: 5.6 s per call under 100k tokens, 9.5 s at 200k to 400k, 13.2 s above 600k. Prefill-dominated calls (under 400 output tokens) go from 3.8 s to 7.7 s across the same range. At the same context, Opus answers in roughly 70 % of Fable's time. So:

- A typical drafting or research turn, about nine calls today, should run 35 % to 45 % faster from context alone, and about 2x faster where Opus takes the small directed changes.
- A routine command (build, open, commit, clean) goes from a 39-second median to a few seconds, because the deterministic path makes no model call, or one short call on a cheap model. Call it 5x to 10x on those turns, which are about 40 % of all prompts by count.
- Scheduled triage drops from about 4 to 5 minutes per run to roughly 2 on Sonnet with caps, and runs within a minute of a file landing instead of up to twelve hours later.
- Overall, across the month's mix, roughly half the wall-clock time per unit of work, with the gains concentrated exactly where the slowness is most felt.

Standing constraint (added September 6, 2026). The deployment's practice-area form adapters, the descriptors, blanks and specs in the private module mounted under `modules/`, and the overlay engine that drives them, must keep working through every workstream. Any change to the form engine, descriptor schema, registry, `sc form` or module discovery either fills the existing module descriptors unchanged, verified against the deployment before a merge to `main`, or ships a mechanical migration script and doc with the change. The descriptor contract is treated as a public API. Concretely: the module today holds eleven descriptors and one test, so the gate is a new `sc form check --all` that discovers every descriptor across built-in, `modules/` and `local/` layers, fills each with fixture data, flattens, and renders the geometry preview, run in the deployment against the candidate engine before any merge to `main`. Building that check is part of W0.

4. Sequence and decisions (resolved September 6, 2026)

Order: W0, W1, W2 (watcher and honest sync first), W3, W4, W5-now, then W5-later. W1 and W2 are a week or two together; W3 and W4 a week or two each; W5-later is open-ended.

1. `main` is production; `dev` is the working branch, checked out at `~/code/prosaic_dev`. Decided.
2. Fable stays the drafting default for initial drafts and major changes. Small, directed changes that a reviewing frontier model specifies are implemented on Opus. Routine commands and triage go to a cheap model. Decided.
3. Knowledge-directory ADR direction approved; migration remains gated file by file. Decided.
4. For the owner's own matters, every role may run on any model and any vendor now; no sanitizer gate applies. The privilege boundary in W5-later is built for deployments serving other people, not as a precondition here. Decided.
5. `/standup` runs at 9 am daily by default. Decided.